Robot Ping-Pong Control Under Physics Variation: Domain Randomization and Residual Reinforcement Learning

Julia Novick Jonathan Zhang Mayand Gulati Sagnik Biswas

UC Santa Barbara

March 12, 2026

Abstract

We present a theory-oriented study of residual reinforcement learning for robotic ping-pong under uncertain simulation physics. The controller combines a finite-state-machine and inverse-kinematics baseline (FSM+IK) with a learned Soft Actor-Critic (SAC) residual that provides bounded joint-space corrections. We compare nominal SAC training on fixed physics against robust SAC training with per-episode domain randomization over mass, friction, restitution, and dissipation parameters. Experiments use 20-episode deterministic evaluations at checkpoints (500k, 750k, 1M) under nominal and randomized test physics. Nominal SAC achieves strongest absolute performance (482 mean hits nominal; 328 randomized), while robust SAC exhibits better relative robustness trends at later checkpoints (+25.9% at 750k and +48.5% at 1M from nominal to randomized evaluation). We provide SAC theory background, implementation-level design analysis, quantitative results, failure-mode analysis, and reproducibility details. Code and artifacts are public at https://github.com/MG-05/ping-pong-domain-randomization.

1 Introduction

Dynamic table-tennis control is a challenging robotic benchmark because high-frequency contact events amplify small model errors. In this project, policy quality is not determined solely by nominal return; it is also determined by resilience under physically plausible perturbations. This motivates a robustness-first framing.

Our system uses a hybrid architecture:

- an analytic inner loop (FSM+IK) for structured intercept and strike behavior,
- a learned residual outer loop (SAC) for corrective adaptation.

This decomposition supports a rigorous question: when does residual learning improve performance, and how does domain randomization change generalization behavior?

We evaluate three research questions:

- **RQ1 (Absolute Performance):** Can residual SAC exceed the hand-engineered baseline?
- **RQ2 (Robustness):** Does domain-randomized training improve behavior under perturbed physics?
- **RQ3 (Stability):** What do variance and survival metrics reveal about policy reliability?

2 Related Work

Entropy-regularized RL. SAC optimizes return plus policy entropy, giving strong continuous-control performance and improved training stability [1].

Continuous-control actor-critic lineage. Deterministic actor-critic methods such as DDPG [2] show the utility of off-policy replay-based learning in robotics, but can be brittle under noise and hyperparameter shifts.

Domain randomization. Randomizing simulator dynamics can reduce overfitting to a single simulator instance and improve out-of-distribution robustness [3, 4].

Robust and residual RL. Robust RL emphasizes performance under perturbations [5]; residual RL combines structured control priors with learned corrections for safety and sample efficiency [6].

Practical implementation framework. Stable-Baselines3 provides reproducible, standard RL training infrastructure that we leverage for SAC training and evaluation orchestration [7].

3 Theoretical Background

3.1 MDP and Residual Action Parameterization

Let $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$ denote the control process [8]. Here, $s_t \in \mathbb{R}^{20}$ contains robot and ball state, and $a_t \in [-1, 1]^7$ is a normalized residual action.

Baseline and residual are combined as

$$q_t^{\text{cmd}} = \Pi_{\mathcal{Q}}\big(q_t^{\text{base}} + \alpha\delta\,\text{clip}(a_t, -1, 1)\big)\,,$$

where α is residual scale, δ is max residual magnitude, and $\Pi_{\mathcal{Q}}$ projects onto joint limits.

3.2 SAC Objective

SAC solves an entropy-regularized objective

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t\left(r_t + \beta\mathcal{H}(\pi(\cdot|s_t))\right)\right].$$

The soft Bellman operator defines

$$\begin{aligned} Q^\pi(s_t, a_t) &= r_t + \gamma\,\mathbb{E}[V^\pi(s_{t+1})]\,, \\ V^\pi(s) &= \mathbb{E}_{a\sim\pi}[Q^\pi(s,a) - \beta\log\pi(a|s)]\,. \end{aligned}$$

3.3 Why Residual SAC Is Appropriate for This Task

Robot ping-pong involves known geometric structure (intercept timing and IK). Learning full control from scratch is unnecessary and increases exploration burden. Residual SAC addresses a narrower problem: improving a feasible baseline policy via bounded corrections.

Proposition 1 (Residual boundedness)**.** *If $q_t^{\text{base}} \in \mathcal{Q}$, then $q_t^{\text{cmd}} \in \mathcal{Q}$ for all t.*

Proof sketch: q_t^{cmd} is explicitly projected by $\Pi_{\mathcal{Q}}$.

Remark 1. *This boundedness ensures residual exploration cannot directly violate joint-position constraints.*

3.4 Robustness Under Transition Shift

Let P_{nom} and P_{rand} denote nominal and randomized transition kernels. Training only on P_{nom} may maximize nominal return while degrading under P_{rand}. Domain-randomized training approximates risk minimization over a distribution of perturbations.

3.5 SAC Training Dynamics in Residual Control

For this project, SAC optimization is best interpreted as learning a correction distribution over a moving baseline controller output. Unlike standard end-to-end policy learning, the action semantics are locally corrective: each action dimension perturbs one joint around a feasible IK-informed command. This affects critic learning in two ways.

First, replay buffer support is naturally concentrated around physically plausible commands, because the baseline controller prevents extreme off-manifold states. Second, value approximation error can still accumulate due to contact discontinuities (ball-paddle impacts), leading to the non-monotonic checkpoint behavior observed in our results.

In actor-critic form, SAC alternates:

- **critic updates**: regress soft Bellman targets under replayed transitions;
- **actor updates**: maximize soft value while preserving entropy;
- **target updates**: Polyak averaging controlled by τ.

Because the reward combines sparse event terms (new hit) and dense shaping terms (alive/apex), critic targets mix abrupt and smooth components. This mixed structure plausibly contributes to temporary value-function mismatch at intermediate checkpoints.

3.6 Entropy Temperature and Exploration Interpretation

In SAC, entropy regularization controls exploration intensity. In a residual-control setting, higher entropy does not mean arbitrary behavior in task space; it means diverse local perturbations around structured baseline commands. This can be advantageous when baseline timing is approximately correct but contact quality requires adaptation.

However, entropy-driven exploration can still produce transiently suboptimal residual patterns before converging to better correction strategies. The 500k $\rightarrow$ 750k dip followed by 1M rebound in both nominal and robust tracks is consistent with such reorganization.

3.7 Critic and Actor Losses (Conceptual Form)

In implementation terms, SAC alternates stochastic gradient steps on:

$$\begin{aligned} \mathcal{L}_Q(\phi) &= \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_\phi(s,a) - y(r,s')\right)^2\right], \\ y(r,s') &= r + \gamma\left(\min_{i\in\{1,2\}} Q_{\bar{\phi}_i}(s',a') - \beta\log\pi_\theta(a'|s')\right), \end{aligned}$$

with $a' \sim \pi_\theta(\cdot|s')$. The actor objective is:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s\sim\mathcal{D},a\sim\pi_\theta}\left[\beta\log\pi_\theta(a|s) - \min_i Q_{\phi_i}(s,a)\right].$$

These objectives are not re-derived in code, but they are the optimization principles inherited from the SAC implementation used by Stable-Baselines3.

3.8 Distributional Robustness Perspective

Let ξ denote randomized physics parameters (mass, friction, restitution, dissipation) sampled from distribution $\mathcal{P}_\Xi$. Nominal training approximates:

$$\max_\pi\ \mathbb{E}_{\tau\sim P_{\xi_0},\pi}[R(\tau)],$$

while robust training approximates:

$$\max_\pi\ \mathbb{E}_{\xi\sim\mathcal{P}_\Xi}\mathbb{E}_{\tau\sim P_\xi,\pi}[R(\tau)].$$

This does not guarantee worst-case optimality, but it better aligns optimization with deployment where physics are uncertain.

Proposition 2 (Expected-risk interpretation)**.** *If test-time dynamics are sampled from $\mathcal{P}_\Xi$, then robust-domain-randomized training is an unbiased Monte Carlo estimator of expected return under that distribution (up to function-approximation error and finite sample effects).*

Practical implication: robust SAC can improve average behavior across perturbations even when it does not maximize peak nominal performance.

4 Methods and Implementation

4.1 Environment and Timing

The environment (`src/envs/residual_env.py`) runs physics at 1 ms and applies RL control at 10 ms. This time-scale separation preserves contact fidelity while keeping policy inference tractable.

4.2 Observation, Action, and Interfaces

Observation (20-D):

$$o_t = [q_{1:7}, \dot{q}_{1:7}, p_b^x, p_b^y, p_b^z, v_b^x, v_b^y, v_b^z].$$

Action (7-D): normalized residual in $[-1, 1]^7$, mapped to bounded joint offsets.

At each control step:

1. evaluate FSM+IK to get q_t^{base},
2. compute residual-corrected command,
3. apply to plant input,
4. advance simulator by control interval.

4.3 Reward Construction

Reward includes:

- alive bonus when ball remains above minimum height,
- hit bonus for debounced contact events,
- apex-tracking bonus $r_{\text{apex}} \exp(-10 \cdot e_{\text{apex}}^2)$,
- drop penalty on terminal failure.

This design balances persistence, contact productivity, and bounce quality.

4.4 Training Configuration

From `src/train_rl.py`, SAC is configured with:

- MLP policy $[256, 256]$,
- learning rate 3×10^{-4},
- batch size 256,
- replay buffer 100k,
- $\gamma = 0.99$, $\tau = 0.005$,
- residual scale 0.5, max residual 0.15 rad.

Two training tracks are used:

- **Nominal SAC:** fixed dynamics.
- **Robust SAC:** randomized dynamics on each reset.

Table 1: Domain randomization ranges used in robust training.

Parameter	Min	Max
Ball mass (kg)	0.0020	0.0035
Ball friction	0.15	0.30
Ball restitution	0.85	0.95
Paddle mass (kg)	0.08	0.15
Paddle friction	0.20	0.60
Paddle restitution	0.80	1.00
Floor dissipation	0.01	0.10
Floor restitution	0.80	1.00

4.5 Domain Randomization Mechanism

`src/utils/randomization.py` samples parameters and writes randomized model/scenario files before each robust episode.

5 Experimental Protocol

5.1 Hypotheses

- **H1:** Residual SAC can exceed FSM baseline hit counts.
- **H2:** Domain-randomized SAC improves relative perturbation robustness.
- **H3:** Learning dynamics are non-monotonic across checkpoints.

5.2 Evaluation Procedure

`src/evaluate.py` runs deterministic policy inference for 20 episodes per model-condition pair. Evaluated checkpoints: 500k, 750k, 1M. Evaluated conditions: nominal and randomized physics.

5.3 Metrics

Definition 1 (Mean hit performance)**.** *For episode hits h_e, $H = \frac{1}{N}\sum_{e=1}^{N} h_e$.*

Definition 2 (Relative perturbation effect)**.**

$$\Delta(\%) = 100 \cdot \frac{H_{\text{rand}} - H_{\text{nom}}}{H_{\text{nom}}}.$$

Definition 3 (Relative variability)**.**

$$\text{CV}(\%) = 100 \cdot \frac{\sigma_{\text{rand}}}{H_{\text{rand}}}.$$

Assumption 1 (Controlled comparisons)**.** *Within each evaluation block, horizon, deterministic inference mode, and episode count are fixed.*

6 Implementation Walkthrough (Conceptual)

6.1 End-to-End Dataflow

At runtime, one control iteration follows this sequence:

1. read simulator outputs: iiwa state, ball state, and ball contact-force proxy;
2. evaluate the FSM controller to generate q_t^{base};
3. query residual SAC policy for normalized action a_t;
4. map a_t to bounded joint corrections and add to q_t^{base};
5. clamp to joint bounds and apply position command to the station;
6. advance simulation by one control interval and compute reward/termination.

This decomposition isolates high-level tactical behavior (intercept/strike logic) from low-level adaptation (residual correction), making failures easier to diagnose than in monolithic policies.

6.2 Why the Observation Space Is Structured This Way

The 20-D observation contains joint position/velocity and ball position/velocity. Conceptually, this is the minimum state needed to:

- infer arm configuration and motion trend,
- infer ball flight state for near-term intercept correction,
- avoid overfitting to simulator-internal artifacts.

No image stream is used; this keeps the problem in a dynamics-control regime rather than perception-heavy policy learning.

6.3 Reward Design Trade-offs

The reward blends sparse success events and dense shaping:

- sparse hit bonuses produce strong task signal but can destabilize critic learning when contacts are rare;
- alive and apex shaping terms reduce reward sparsity but may create local optima (e.g., keeping short rallies alive rather than maximizing long-term hit count);
- drop penalties discourage catastrophic failure but can overweight terminal events if improperly tuned.

In this project, the combined reward appears effective for discovering high-hit trajectories at 1M, but also exhibits checkpoint volatility, suggesting potential future reward-scaling work.

6.4 Contact Event Debouncing and Measurement Semantics

Hit counting uses force-thresholded contact with cooldown debouncing. This matters for interpretation: "hits" are not simply every physics-step contact, but filtered events intended to approximate meaningful paddle-ball interactions. Therefore, reported hit-rate differences can reflect both policy behavior and event-definition sensitivity, a point we include in validity threats.

6.5 Training Infrastructure and Reproducibility

The training loop uses a standard set of RL engineering controls:

- checkpoint callback for temporal model snapshots,
- eval callback for periodic deterministic policy evaluation,
- monitor-wrapped environments for episodic metrics,
- fixed seeds to improve experiment repeatability.

This scaffolding is crucial in graduate-level projects because it enables post-hoc analysis of dynamics such as the 750k dip.

6.6 Domain Randomization as Scenario Generation

Randomization is implemented by sampling parameters and writing randomized model/scenario files used at environment reset. This means perturbations alter actual simulation artifacts (not merely internal coefficients) and better reflect realistic structural variation in contact dynamics.

6.7 FSM Inner Loop: Why It Matters for Residual Learning

The FSM controller provides a sequence of task-relevant phases (e.g., intercept planning, strike timing, follow-through). Conceptually, this contributes three benefits to residual learning:

- **Trajectory feasibility prior:** IK-generated targets keep the arm in physically meaningful postures.
- **Temporal prior:** strike windows are aligned with predicted ball intercept timing.
- **Contact prior:** the controller already encodes an intended contact geometry.

As a result, residual SAC is not learning table tennis from scratch; it is learning *how to improve* an already competent policy.

6.8 IK and Planning Constraints (Conceptual)

Even without exposing all solver internals in this paper, the implementation reflects a constrained optimization structure: solve for arm configurations near a seed joint state while satisfying kinematic and timing constraints.

Table 2: FSM baseline robustness (dedicated baseline report).

Condition	Mean Hits	Full 20 s Survival
Deterministic init	154 (single run)	Yes
Noisy init (±4 cm)	112.3 ± 46.7	25/50 (50%)
Stress init (±8 cm)	104.3 ± 54.9	23/50 (46%)

This produces smooth command sequences that are then corrected by residual actions.

In theory language, the baseline maps state to a low-dimensional manifold of plausible commands. The residual policy explores locally around this manifold. This is one reason residual RL can be data-efficient in robotics.

6.9 Why Deterministic Evaluation Is Used

Evaluation uses deterministic action selection to isolate policy quality from stochastic action sampling noise. This is particularly important when comparing checkpoints, because otherwise performance differences can be confounded by changing policy entropy realization.

However, deterministic evaluation also creates a limitation: under nominal fixed physics, some metrics become near-deterministic (e.g., near-zero standard deviation), reducing visibility into variability. We therefore include randomized evaluation and variance metrics to recover uncertainty information.

6.10 MuJoCo Transfer Scaffold in Project Scope

The repository includes a MuJoCo transfer scaffold (`mujoco_transfer/`) with FSM/IK execution and metric logging. In this project phase, it primarily serves as transfer infrastructure rather than a full second-domain RL benchmark.

This scaffold is still scientifically useful: it separates the question "does the control stack work in Drake?" from "how sensitive is it to simulator implementation differences?" and sets up future sim-to-sim transfer experiments.

7 Results

7.1 FSM Baseline Robustness

The baseline controller is strong and should not be treated as a trivial reference.

7.2 Main Quantitative Comparison

Finding 1 (Absolute performance): Nominal SAC 1M is strongest in raw hit count.

Finding 2 (Relative robustness): Robust SAC becomes positive in Δ at later checkpoints.

Finding 3 (Learning dynamics): Both tracks exhibit a 750k dip and 1M rebound.

Finding 4 (Consistency): Nominal SAC 1M has high randomized variance ($\sigma = 106$); robust SAC 1M is lower ($\sigma = 30.9$).

7.3 Extended Metric Analysis

To separate rally quantity from episode duration, we compute mean hit throughput $\rho = H/T$, where T is mean simulation time.

This table clarifies a subtle behavior: robust policies can exhibit short episodes with very high instantaneous hit throughput. Thus, high hit count and long survival are related but not identical objectives in this setup.

7.4 Checkpoint-by-Checkpoint Interpretation

500k stage. Nominal 500k is already competitive with baseline in throughput, while robust 500k remains conservative in absolute hit count. This is consistent with robust training paying an early tax for invariance learning.

750k stage. Both nominal and robust trajectories dip in nominal evaluation. Robust 750k is notable because randomized performance exceeds nominal performance (+25.9%), suggesting adaptation to the randomization distribution even before absolute performance matures.

1M stage. Nominal 1M undergoes a clear breakthrough in absolute hits under both conditions but retains high OOD variance. Robust 1M improves both mean hits and relative perturbation behavior but still trails nominal 1M in absolute peak.

7.5 Cross-Metric Tensions

The results expose three objective tensions:

- **Peak hits vs perturbation robustness:** nominal 1M dominates one, robust 1M trends better on the other.
- **Throughput vs episode duration:** some policies hit rapidly in shorter windows.
- **Mean vs predictability:** higher mean can coexist with larger variance.

8 Failure Analysis and Lessons

The original goal was a single policy that is best in absolute performance and robustness. Under the present budget, this did not happen. Two failure modes emerged:

Failure mode A: Nominal over-specialization. Nominal SAC learns high-return trajectories but degrades under perturbation (−32.0% at 1M). This is consistent with specialization to fixed contact dynamics.

Table 3: 20-episode evaluation summary across models and physics conditions.

Model	Eval Physics	Mean Hits	Std Hits	Max Hits	Mean Sim Time (s)	Δ vs Nominal
FSM Baseline	Nominal	159.0	0.0	159	20.0	–
FSM Baseline	Randomized	145.3	29.0	189	17.7	−8.6%
Nominal SAC 500k	Nominal	144.0	0.0	144	11.0	–
Nominal SAC 500k	Randomized	112.8	55.1	274	8.5	−21.7%
Nominal SAC 750k	Nominal	110.0	0.0	110	6.6	–
Nominal SAC 750k	Randomized	61.3	35.9	156	3.9	−44.3%
Nominal SAC 1M	Nominal	**482.0**	0.0	**482**	20.0	–
Nominal SAC 1M	Randomized	**328.0**	**106.0**	**474**	13.9	−32.0%
Robust SAC 500k	Nominal	53.0	0.0	53	3.5	–
Robust SAC 500k	Randomized	43.4	11.0	66	2.8	−18.2%
Robust SAC 750k	Nominal	23.0	0.0	23	1.7	–
Robust SAC 750k	Randomized	29.0	7.5	51	1.8	+25.9%
Robust SAC 1M	Nominal	76.0	0.0	76	1.7	–
Robust SAC 1M	Randomized	112.9	30.9	181	2.7	+48.5%

Table 4: Derived hit throughput (hits/s) from evaluation means.

Model	Nominal ρ	Randomized ρ
FSM Baseline	7.95	8.23
Nominal 500k	13.10	13.28
Nominal 750k	16.57	15.80
Nominal 1M	24.10	23.60
Robust 500k	15.19	15.25
Robust 750k	13.37	16.05
Robust 1M	45.24	41.50

Failure mode B: Robust under-optimization. Robust SAC improves relative robustness but remains below nominal SAC in peak absolute hits at 1M.

Interventions attempted. We increased the training budget to 1M and retained intermediate checkpoint evaluation; used wide randomization ranges over contact and inertial parameters; and compared both training types under both evaluation conditions.

Interpretation. Domain-randomized training appears to allocate capacity toward invariance across physics regimes, which may delay convergence to highly specialized nominal strategies.

9 Threats to Validity

- **Single-seed training**: uncertainty across random initializations is not fully characterized.
- **Deterministic nominal evaluation**: near-zero nominal std is expected but limits uncertainty reporting.
- **Simulation-only conclusions**: sim-to-real transfer is not yet validated on hardware.
- **Metric coupling**: hit counting and horizon settings influence rankings.

10 Model-Family Case Analysis

10.1 Nominal SAC Family

The nominal SAC checkpoints illustrate a specialization trajectory. At 500k and 750k, the policy has moderate hit throughput and strong degradation under perturbation. At 1M, the policy reaches a qualitatively different regime with very high hit counts in both nominal and randomized tests, but with substantial variance under randomized physics.

This pattern suggests the learned strategy eventually discovers a high-frequency control mode that strongly exploits nominal simulator structure. The same strategy can still perform well on average under perturbation, but instability increases, consistent with brittle contact sensitivity.

10.2 Robust SAC Family

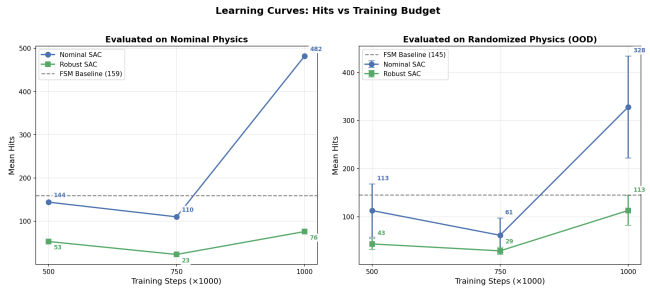
The robust SAC family behaves differently. Early checkpoints are conservative and low in absolute hits, then later checkpoints improve relative perturbation behavior (positive Δ). This can be interpreted as the policy gradually internalizing dynamics-invariant corrective behavior.

Although robust 1M underperforms nominal 1M on absolute hit count, it is less volatile under randomized physics. For applications prioritizing reliability under model mismatch, this may be preferable despite lower peak returns.

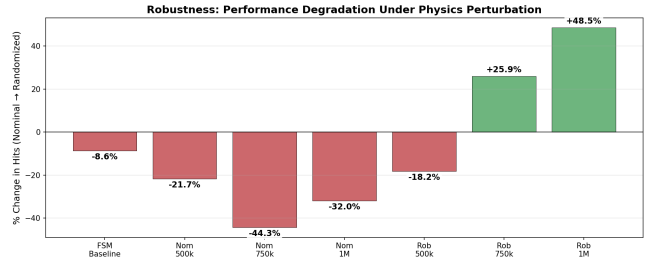
10.3 Baseline vs Learned Controllers

The FSM+IK baseline remains a useful anchor because it is:

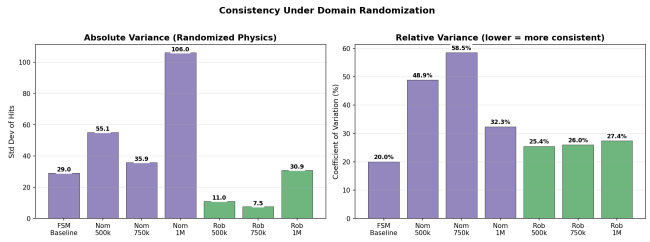
- interpretable (state-machine phases and IK objectives are inspectable),
- stable in nominal deployment,



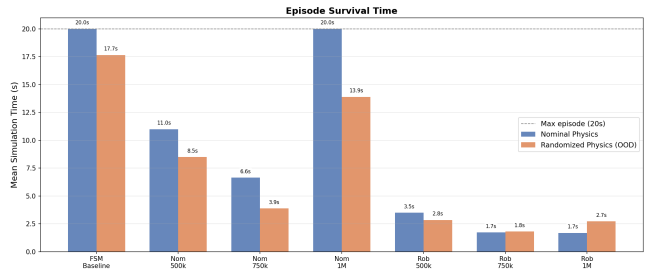
(a) Learning curves versus training budget.



(b) Relative perturbation effect Δ.



(c) Absolute and relative variability under randomized physics.



(d) Episode duration comparison.

Figure 1: Core experimental results: performance curves, robustness degradation, variance profiles, and episode survival times across all models and evaluation conditions.

- reasonably robust under moderate perturbation.

Residual learning should therefore be judged not only by raw score uplift, but by whether it preserves or improves these baseline reliability properties.

10.4 Practical Selection Guidance

Given current evidence, a deployment-oriented selection rule could be:

- choose **Nominal 1M** when maximizing expected throughput in environments close to nominal simulator dynamics;
- choose **Robust 1M** when dynamics uncertainty is large and performance consistency is prioritized;
- retain **FSM+IK** as a fallback safety baseline.

This policy-selection framing is often more actionable than a single "best model" declaration.

11 Reproducibility and Experimental Governance

11.1 Data Provenance

The quantitative values in this paper are taken from repository artifacts:

- `results/all_eval_results.json`
- `results/fsm_baseline_report.md`
- `results/rl_training_report.md`
- generated figures in `results/final_*.png`

This provenance path makes each table and plot auditable.

11.2 Evaluation Integrity Checks

A graduate-level evaluation should explicitly check:

- deterministic-evaluation flag consistency across runs,
- checkpoint naming and step-count alignment,
- fixed episode count across compared rows,
- identical metric definitions (especially hit event detection),
- correct mapping between plot labels and table rows.

These checks are straightforward but essential for avoiding silent comparison errors.

11.3 Recommended Reporting Standard

For stronger final claims, future iterations should report:

- multi-seed mean and confidence intervals,
- per-episode distributions (not only mean/std aggregates),
- sensitivity to reward weights and residual envelope,
- additional robustness stress tests beyond current randomization ranges.

Even when project goals are only partially met, this reporting standard strengthens scientific credibility and aligns with the syllabus emphasis on rigorous analysis over merely positive outcomes.

12 Future Directions

- multi-seed confidence intervals for all checkpoints/conditions;
- curriculum randomization schedules (narrow-to-wide);
- risk-sensitive objectives targeting lower-tail performance;
- ablations on residual scale/max residual envelope;
- MuJoCo transfer plus real-robot calibration pipeline.

13 Conclusion

This project demonstrates a rigorous hybrid-control study: residual SAC significantly improves baseline performance, while domain randomization changes robustness behavior in a measurable way. The main scientific outcome is a data-backed trade-off between nominal specialization and perturbation robustness, rather than a single universally dominant policy.

Code Availability and Reproducibility

All code, trained models, and evaluation artifacts are publicly available at https://github.com/MG-05/ping-pong-domain-randomization. To reproduce the full experimental pipeline (training, evaluation, and report generation), run the following commands with the repository dependencies installed:

```
python -m src.train_rl --steps 1000000 \
  --no-randomize \
  --save-path data/sac_nominal_1m --seed 42

python -m src.train_rl --steps 1000000 \
  --randomize \
  --save-path data/sac_robust_1m --seed 42

python scripts/run_all_evals.py
python results/generate_final_report.py
```

References

[1] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of ICML*, 2018.

[2] Timothy Lillicrap, Jonathan Hunt, Alexander Pritzel, et al. Continuous Control with Deep Reinforcement Learning. arXiv:1509.02971, 2015.

[3] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In *Proceedings of IROS*, 2017.

[4] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. In *Proceedings of ICRA*, 2018.

[5] Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust Adversarial Reinforcement Learning. In *Proceedings of ICML*, 2017.

[6] Tobias Johannink, Shikhar Bahl, Ashvin Nair, et al. Residual Reinforcement Learning for Robot Control. In *Proceedings of ICRA*, 2019.

[7] Antonin Raffin, Ashley Hill, Adam Gleave, et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. https://github.com/DLR-RM/stable-baselines3, 2021.

[8] Richard Sutton and Andrew Barto. Reinforcement Learning: An Introduction. MIT Press, 2nd edition, 2018.